DEBUGGING · DOMAIN 9 - DEBUGGING AND DEPLOYING

Bundles and Git-based delivery

Section 9. Deploying resources, deploying code, and knowing which you are doing.

SECTION 9 of 10 WEIGHT 10% TOPIC CI/CD

admin@sthithapragna.com

A bundle deploys **resources**: jobs, pipelines and their configuration. A Git folder deploys **code**. Those are different problems.

Asset Bundles are now Declarative Automation Bundles

- The documentation now says **Declarative Automation Bundles, formerly known as Databricks Asset Bundles**.
- **The acronym is unchanged**, and so is the command line. You still run **databricks bundle**.
- The exam guide uses the new name with the old one in brackets, so it has absorbed this one.
- Older material, including most practice banks, uses only the old name.

> ON THE EXAM
>
> Unusually, this is a rename the exam guide did catch. Expect either name in a question and treat them as identical.

The commands you will meet

COMMAND	WHAT IT DOES
`bundle init`	Create a new bundle project from a template.
`bundle validate`	Check the configuration and show what would be deployed.
`bundle deploy`	Create or update the resources in the workspace for a target.
`bundle run`	Run a job or pipeline defined in the bundle.
`bundle destroy`	Remove the resources the bundle previously deployed.

ON THE EXAM

Every one of them takes a target. The target is what makes the same source deploy differently to dev and production.

Three behaviours you get for free

- **Prefixes every resource** that is not a file or notebook with a dev marker and your short name, and **tags jobs and pipelines** as dev.
- **Marks all deployed pipelines as development**, which is module 5's development mode.
- **Enables the compute override flag**, letting you point the whole bundle at one existing cluster.
- That flag **overrides any and all existing cluster definitions** in the configuration. It is deliberately blunt.

> ON THE EXAM
>
> The prefix is what lets a whole team deploy the same bundle into one workspace without colliding. Each developer gets their own copy.

Two validations, and one you can override

- **Validates that every deployed pipeline is marked development false.** A dev pipeline cannot reach production by accident.
- **Validates that the current Git branch matches the branch named in the target**, if you named one.
- That branch check **can be overridden with a force flag**, which is worth knowing and worth using rarely.
- Unlike development mode, **production mode does not allow overriding** its behaviours.

> ON THE EXAM
>
> The branch validation is a guardrail against deploying to production from a feature branch, which is exactly the accident it sounds like.

What changes if you do not use a service principal

- Databricks recommends service principals for production, enforced by setting the run-as identity.
- **Without one, production mode applies two extra validations.**
- That the artifact, file, root and state paths are **not overridden to a specific user**.
- And that the **run-as and permissions mappings are specified**, so it is explicit who owns and can use the deployment.

> ON THE EXAM
>
> Module 24 explained why: a job runs with its owner's permissions, so a production job owned by a person carries that person's access.

Resource beats preset beats mode

- A **mode** supplies default behaviours. **Presets** customise them. **Individual resource settings** override the presets.
- Documented example: a job with a maximum of ten concurrent runs, under a preset of twenty, keeps **ten**.
- Another: a schedule set to unpaused, under a preset that pauses schedules, stays **unpaused**.
- So the most specific setting always wins, which is the behaviour you would hope for.

> ON THE EXAM
>
> Worth knowing precisely, because a preset that appears to have no effect is usually being overridden by an explicit resource setting.

Deploy to an immutable folder

- Databricks recommends deploying a bundle to an **immutable, read-only folder**.
- That guarantees deployed assets **cannot be modified by non-admin users**.
- And that **running jobs are not affected by an in-progress deployment**.
- Without it, somebody editing a deployed notebook creates drift that the next deploy silently reverses.

> ON THE EXAM
>
> The second reason is the one people forget. A deployment landing mid-run should not change what the running job is executing.

The stated division of labour

- **Use Git folders for interactive development.** That is the documentation's own sentence.
- **For CI/CD and production deployments, use bundles**, with versioned artifacts and workload identity federation.
- If you prefer to deploy **only code** into the workspace, a production Git folder is the supported alternative.
- So Git folders in production are a narrower answer, not a wrong one.

> ON THE EXAM
>
> Note what the alternative is scoped to: deploying only code. If you also need jobs and pipelines defined as code, that is a bundle.

User-level and production Git folders

- **User-level folders** live under the users path and act as **local checkouts** where developers work and push.
- **Production folders** are created by **admins, outside user folders**, and contain production deployment branches.
- A production folder **should only be updated programmatically when a pull request is merged**.
- Creating one requires a **service principal with a configured Git credential** for the repository.

> ON THE EXAM
>
> Two objects with the same name doing opposite jobs. One is a workspace for a person; the other is a deployment target nobody edits.

Who can do what to a production Git folder

- **Limit user access to run-only.** People need to run workflows, not to edit the checkout.
- **Allow only admins and service principals to edit.**
- Optionally grant view to all workspace users, to support discovery.
- Any human able to edit a production Git folder can put code into production without a pull request.

> ON THE EXAM
>
> Module 24's least privilege, applied to a deployment target. Edit access here bypasses your entire review process.

What Git folders will not do

- **No SSH support**, and **no commit signing**.
- **Sparse checkout cannot be disabled once enabled.**
- Databricks **recommends against Git folders backed by monorepos**. Cloning one can exceed memory and disk limits.
- For a repository holding many projects, **split it, or use sparse checkout** to limit which directories are cloned.

> ON THE EXAM
>
> The monorepo warning is specific and practical. Sparse checkout is the escape hatch, and it is a one-way decision.

They are not alternatives

- Databricks' own tip: **define jobs and pipelines in source using bundles, then manage those bundles in workspace Git folders**.
- Bundles in the workspace let a team develop and deploy a bundle from a Git folder in the interface, **without the command line**.
- That opens bundle-based delivery to analysts and scientists who were never going to install a CLI.
- So the exam's two objectives describe two halves of one workflow.

> ON THE EXAM
>
> The framing to carry: Git folders for how the code gets there and gets reviewed, bundles for what gets created in the workspace.

What to carry into the exam

- **Bundles deploy resources. Git folders deploy code.** The docs say use Git folders for interactive development.
- Development mode **prefixes and tags**, marks pipelines as development, and enables the compute override.
- Production mode **validates** pipelines are not development, and that the Git branch matches. Force overrides the branch check.
- Without a service principal, production mode adds **path and permissions validations**.
- Precedence is **resource, then preset, then mode**. Production Git folders are edit-restricted to admins and service principals.

> ON THE EXAM
>
> That completes Section 9 at 10%. Section 10, data modeling, is next and is the last teaching section at 6%.

> Six engineers work on the same bundle. When two deploy to the shared development workspace at once, they expect their jobs to overwrite each other. In practice each sees only their own copies.
>
> Their target sets mode to development.

Why do the deployments not collide?

A Development mode prefixes resources with a dev marker and the deploying user's short name, so each engineer gets their own set.

B Bundles lock the workspace during deployment, so deploys are serialised.

C Each deployment creates a separate workspace automatically.

D Development mode deploys to the user's home folder, which is why they are isolated.

◆ PAUSE AND DECIDE

Why do the deployments not collide?

A Development mode prefixes resources with a dev marker and the deploying user's short name, so each engineer gets their own set.

The documented behaviour, and precisely what makes a shared development workspace usable by a team.

B Bundles lock the workspace during deployment, so deploys are serialised.

There is no such lock, and serialising would not prevent overwriting.

C Each deployment creates a separate workspace automatically.

Workspaces are not created by a deploy.

D Development mode deploys to the user's home folder, which is why they are isolated.

Paths do differ per user, and the resource naming prefix is the documented mechanism for this.

ON THE EXAM

The prefix looks cosmetic and is doing real work. It is what makes one shared development workspace serve a whole team.

> A deployment to the production target fails validation. The engineer is on a feature branch, and the target specifies the main branch. They need to deploy urgently to fix a broken pipeline.
>
> A colleague suggests removing the branch specification from the target.

What is the better response?

A. Remove the branch specification, since it is blocking an urgent fix.

B. Deploy with the force flag for this one deployment, then merge the fix to main properly.

C. Switch the target to development mode, which skips the branch validation.

D. The deployment cannot proceed until the pull request is merged; there is no override.

◆ PAUSE AND DECIDE

What is the better response?

A. Remove the branch specification, since it is blocking an urgent fix.

Removes the guardrail permanently to solve a one-off problem. Nobody puts it back.

B. Deploy with the force flag for this one deployment, then merge the fix to main properly.

The documented override, scoped to one deployment, leaving the guardrail in place.

C. Switch the target to development mode, which skips the branch validation.

Would also prefix every resource and mark pipelines as development, changing production.

D. The deployment cannot proceed until the pull request is merged; there is no override.

The branch validation is explicitly overridable with force.

ON THE EXAM

A and B reach the same place. One removes the guardrail for everyone forever; the other bypasses it once and leaves it standing.

> A team sets a preset on their production target pausing all job schedules, so that deployments never start work unexpectedly. After deploying, they find one job's schedule is still running.
>
> That job's definition explicitly sets its schedule to unpaused.

What is happening?

A Presets override individual resource settings, so the preset should have won; this is a bug.

B Individual resource settings override presets, which override mode defaults. The explicit unpaused setting wins.

C Schedule pausing is only supported in development mode.

D The preset applies only to jobs created after it was added.

◆ PAUSE AND DECIDE

What is happening?

A Presets override individual resource settings, so the preset should have won; this is a bug.

The precedence runs the other way. Resource settings are the most specific and win.

B Individual resource settings override presets, which override mode defaults. The explicit unpaused setting wins.

Documented precedence, with this exact example given in the docs.

C Schedule pausing is only supported in development mode.

Presets apply to any target.

D The preset applies only to jobs created after it was added.

Presets apply at deployment, not at creation.

ON THE EXAM

This exact case is one of the documentation's own examples. The most specific setting always wins, which is usually what you want and occasionally not.

> A production bundle is deployed under a named engineer's identity rather than a service principal. Deployment succeeds, but production mode reports additional validation checks the team did not expect.
>
> They ask why their colleague's identical bundle in another workspace does not show them.

What is the explanation?

A. Production mode applies extra path and permissions validations when you do not deploy as a service principal.

B. The other workspace is on an older CLI version.

C. Validation behaviour depends on the workspace's pricing tier.

D. The extra checks appear only on first deployment of a bundle.

◆ PAUSE AND DECIDE

What is the explanation?

A Production mode applies extra path and permissions validations when you do not deploy as a service principal.

Documented: without a service principal, production mode additionally validates the paths and the run-as and permissions mappings.

B The other workspace is on an older CLI version.

Possible in general, and the identity difference is the documented explanation.

C Validation behaviour depends on the workspace's pricing tier.

Deployment mode behaviour is not tier-dependent.

D The extra checks appear only on first deployment of a bundle.

They depend on the deploying identity, not on how many times you have deployed.

ON THE EXAM

The extra validations are a nudge. Databricks would rather you used a service principal, and checks more carefully when you do not.

> A team clones their organisation's monorepo into a Git folder so that all projects are available in one place. Cloning is slow, some operations time out, and the folder occasionally fails to sync.
>
> The repository holds around forty projects.

What should they do?

A. Increase the cluster size, since Git operations run on the attached compute.

B. Databricks recommends against monorepo-backed Git folders. Split the repository, or use sparse checkout to clone only what is needed.

C. Switch to SSH, which handles large repositories better.

D. Disable sparse checkout, which is causing the sync failures.

◆ PAUSE AND DECIDE

What should they do?

A Increase the cluster size, since Git operations run on the attached compute.

The limits are on the Git folder, not on your cluster.

B Databricks recommends against monorepo-backed Git folders. Split the repository, or use sparse checkout to clone only what is needed.

Stated directly: cloning a monorepo can exceed Git folder memory and disk limits and slow operations.

C Switch to SSH, which handles large repositories better.

Git folders do not support SSH at all.

D Disable sparse checkout, which is causing the sync failures.

Sparse checkout is the remedy here, and it cannot be disabled once enabled anyway.

ON THE EXAM

D inverts the answer and also names something that cannot be done. Sparse checkout is a one-way decision.

> A team version-controls all their notebooks in a production Git folder, updated automatically when pull requests merge. Job definitions, schedules and permissions are still configured by hand in the workspace UI.
>
> After an incident, they cannot reconstruct what a job's configuration was last month.

What is missing?

A. Nothing structural; they should enable audit logging to reconstruct configuration history.

B. Git folders deploy code, not resources. Defining jobs and pipelines in a bundle puts their configuration under version control too.

C. They should move the production Git folder to a user folder so history is retained.

D. Job configuration cannot be version controlled on Databricks.

◆ PAUSE AND DECIDE

What is missing?

A Nothing structural; they should enable audit logging to reconstruct configuration history.

Audit records who changed something. It is not a substitute for the configuration being in source control.

B Git folders deploy code, not resources. Defining jobs and pipelines in a bundle puts their configuration under version control too.

The distinction this module opened with, arriving as a real consequence.

C They should move the production Git folder to a user folder so history is retained.

User-level folders are development checkouts, and the location is not the problem.

D Job configuration cannot be version controlled on Databricks.

That is exactly what bundles are for.

ON THE EXAM

Half the deployment is versioned and half is not. The half that is not is where the incident came from.